Draw it or Lose It

**CS 230 Project Software Design Template**

Version 1.0

**Table of Contents**

**CS 230 Project Software Design Template** 1

**Table of Contents** 2
**Document Revision History** 2
**Executive Summary** 3
**Requirements** 3
**Design Constraints** 3
**System Architecture View** 3
**Domain Model** 3
**Evaluation** 4
**Recommendations** 5

## Document Revision History

| Version | Date | Author | Comments |
|---|---|---|---|
| 1.0 | 03/20/2026 | Duane Day Jr | Initial version of software design document |

**Instructions**
Fill in all bracketed information on page one (the cover page), in the Document Revision History table, and below each header. Under each header, remove the bracketed prompt and write your own paragraph response covering the indicated information.

## Executive Summary

The purpose of this project is to design a web based version of Draw It or Lose It for The Gaming Room. The client wants the game to be accessible across multiple platforms such as desktops and mobile devices. The system will use a distributed web based design so users can play without installing software. This improves accessibility and performance. The system will also be scalable to support more users over time. Security and reliability will be important throughout development.

## Requirements

The system must allow multiple teams and players to participate in a shared game session. Each game, team, and player must have a unique identifier to prevent duplication. The system must support multiple operating systems including Windows, Mac, Linux, and mobile devices. It must allow real time interaction between users. From a technical standpoint, the system must be scalable, secure, and efficient. It should follow object oriented programming principles and support cloud based deployment. Data must remain consistent and reliable at all times.

## Design Constraints

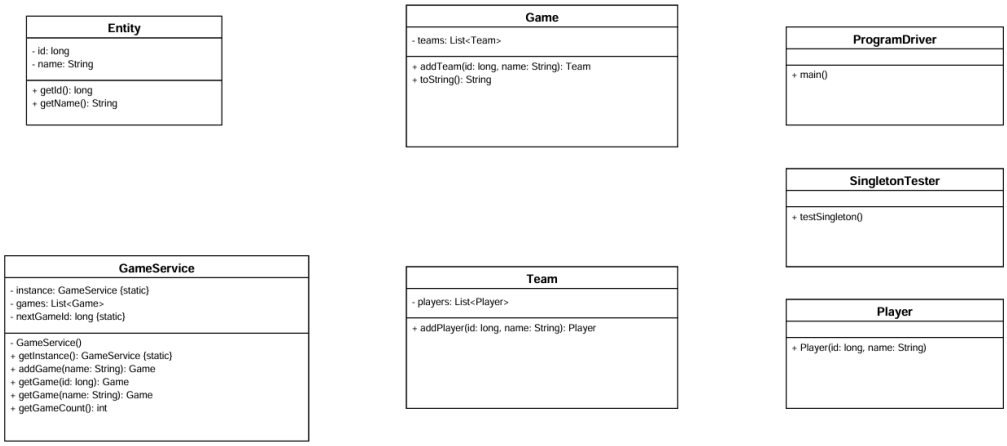
The application must run in a web based distributed environment. This means the system must handle multiple users at the same time without performance issues. It must be compatible across different platforms, which limits the technologies that can be used. Security is also a constraint because user data must be protected. Network reliability may affect performance, so the system must handle connection issues. These constraints require careful design, testing, and the use of scalable solutions like cloud services.

## System Architecture View

The system will use a client server architecture. The client side includes web browsers and mobile devices used by players. The server side handles game logic, user data, and communication between users. A distributed system allows multiple servers to handle requests efficiently. A centralized database stored all game data.

## Domain Model

## UML Diagram




The UML diagram represents the structure of The Gaming Room application and shows how the main classes interact with each other. The core classes include GameService, Game, Team, Player, Entity, ProgramDriver, and SingletonTester.

The Entity class is the base class that contains common attributes such as id and name. This class is inherited by Game, Team, and Player, demonstrating inheritance and reducing duplicate code.

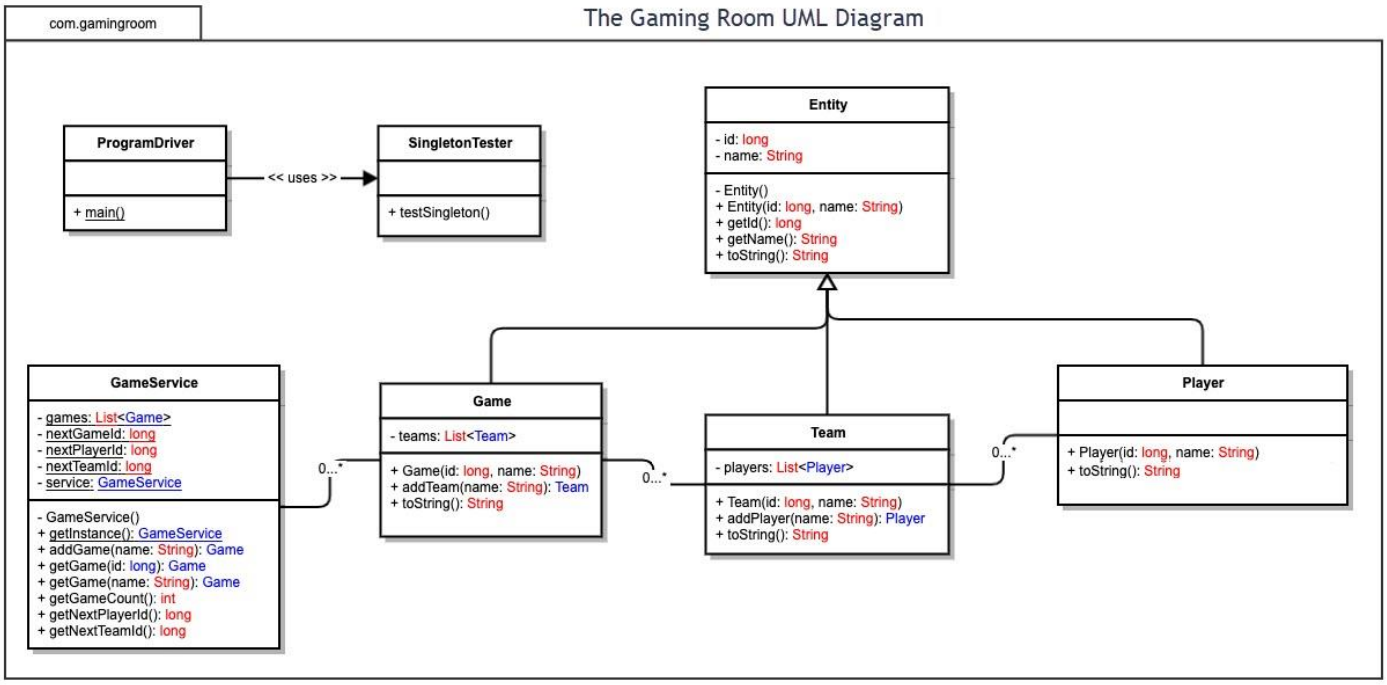
The GameService class acts as a central manager. It stores a list of games and ensures that each game, team, and player has a unique ID. This demonstrates encapsulation since data is controlled through methods.

The Game class contains multiple teams, showing a one to many relationship. The Team class contains multiple players, also showing a one to many relationship. These relationships show how data flows through the system.

The Player class represents individual users, while the Team class groups players together.

ProgramDriver and SingletonTester are used to test the system and verify that only one instance of GameService exists.

This design demonstrates object oriented principles such as inheritance, encapsulation, and abstraction. These principles help make the system efficient, organized, and scalable.




## Evaluation

In each cell, remove the bracketed prompt and write your own paragraph response covering the indicated information.

| Development Requirements | Mac | Linux | Windows | Mobile Devices |
|---|---|---|---|---|
| **Server Side** | Mac can host web applications but is not commonly used for large scale server environments. It is stable and secure but more expensive. It is better suited for development than production hosting. | Linux is the best choice for server hosting. It is open source, cost effective, secure, and widely used in cloud environments. It offers strong performance and scalability. | Windows servers are user friendly and integrated well with Microsoft tools. However, they require licensing costs and more resources. They are suitable but not as efficient as Linux. | Mobile devices are not suitable for server side hosting due to limited processing power and reliability. |
| **Client Side** | Mac provides a smooth and reliable user experience. It supports modern browsers but can be more expensive. | Linux is flexible and cost-effective but may require more technical knowledge for users. | Windows is widely used and compatible with most software. It is easy to use and a strong choice for clients. | Mobile devices require responsive design and optimization for different screen sizes and performance limits. |
| **Development Tools** | Tools include Xcode, IntelliJ, and VS Code for development. | Tools include Eclipse, VS Code, and Git for open source and server development. | Tools include Visual Studio and VS Code. These are widely used and beginner friendly. | Tools include Android Studio and Xcode. Web frameworks can also support cross platform development. |

**Recommendations**

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: Linux is the best operating platform because it is secure, cost effective, and widely used for web applications. It supports scalability and cloud deployment. Since Linux is open source, it reduces licensing costs and allows more flexibility when configuring servers. This makes it ideal for hosting a distributed web based game like Draw It or Lose It.

2. **Operating Systems Architectures**: A client server architecture should be used with distributed servers. This allows multiple users to connect at the same time and improves performance and reliability. This architecture ensures that many users can connect at the same time without slowing down the system.

3. **Storage Management**: A cloud based database should be used to store game data. This ensures consistency, backup, and easy access across platforms. Cloud storage also provides automatic backups and improves data availability. Using a centralized database ensures that all players and teams have up to date information during gameplay.

4. **Memory Management**: The system should use efficient memory management to handle multiple users. Dynamic allocation helps improve performance and prevent crashes. This improves performance and prevents crashes. Proper memory management ensures smooth gameplay and efficient use of system resources.

5. **Distributed Systems and Networks**: The system should use cloud networking to allow communication across platforms. It must handle connection issues and ensure smooth performance. Using distributed systems improves reliability and ensures the application remains available even if one server fails.

6. **Security**: Security should include encryption, secure login systems, and access control. User data must be protected during transmission and storage. Linux provides strong security features. Implementing HTTPS and authentication methods helps prevent unauthorized access; this ensures everything will remain secure.